

Birla Institute of Technology and Science, Pilani
CS F211 (Data Structures and Algorithms), Lab Test, OPEN BOOK

April 06, 2025

Time: 2.5 Hrs

MM: 60

Note: Download the folder (DSA Lab Test 2025). Rename it with <ID Number_FirstName>. Inside the folder, you are given *three .h files, four .c files, one .txt file, and one driver (i.e. main.c) file*. As per the tasks given below, modify the code within these files only. Make sure that you are saving your work and not overwriting it. It is solely your responsibility to save your files and upload them correctly.

You are given a text file named `students.txt` containing records of students. Each record consists of the following fields: **BITS_ID**, **Name**, and **CGPA**, in the format shown below:

```
2021A7PS0101P Ani 9.2
2020B1A30020P Kartik 8.7
2019A1PS1113P Saumya 7.5
2015B2A70004P Anirban 8.9
...
```

Format of BITS_ID: The **BITS_ID** is a 13-character string with the following components:

- **Characters 1–4:** Year of admission (when treated as an integer)
- **Characters 5–8:** Branch code (these characters can be any combination of alphabets and digits, e.g. A3B4, BBAA, A1PS, B2TS, B255, etc.).
- **Characters 9–12:** Roll number (when treated as an integer)
- **Character 13:** Always 'P'

Your Tasks:

You are required to complete the following tasks (Task-1 to Task-3). Some functions are already provided to you; your job is to implement the remaining ones.

Task-1: Reading Records

- Read all student records from `students.txt` into a dynamically allocated array of `struct Student`.
- Start by allocating memory for 50 student records.
- If more records are read, increase the array size in blocks of 30 records as needed.
- *If you cannot read using dynamic memory allocation, you can just read the given 100 records of students.txt file in struct Student array. Of course, you will loose some marks for this.*

Files Provided:

- `student.h` (gives definition of struct Student and function declarations)
- `student.c` (You need to implement the function to read the data. Other helper functions are also available in this file.)

Task-2: Sorting Records

- Sort the student records using **insertion sort** in ascending order based on the following criteria (**separately**):
 - ROLL NUMBER (Characters 9–12 of `BITS_ID`)
 - BRANCH (Characters 5–8 of `BITS_ID`)

[Note: You should not use any built in sorting functions. You have to implement Insertion sorting only.]

Files Provided:

- `sorting.h`
- `sorting_branch.c` – Modify the given CGPA-based insertion sort, to sort the student records by **BRANCH** of **BITS_ID**

For example, if the BITS_ID numbers are: 2023A7PS1201P, 2025A1B51300P, 2024A3PS1205P, then their sorted order by BRANCH is 2025A1B51300P, 2024A3PS1205P, 2023A7PS1201P.

- `sorting_roll.c` – Similarly, modify the CGPA-based insertion sort, to sort the student records by **ROLL NUMBER** of **BITS_ID**.

For example, if the BITS_ID numbers are: 2022A1B51300P, 2023A3PS1205P, 2025A7PS1201P, then their sorted order by ROLL NUMBER is 2025A7PS1201P, 2023A3PS1205P, 2022A1B51300P.

Task-3: Hashing

Implement hashing to insert the sorted student records into a hash table.

- Create a hash table of fixed size.
- Use **linear probing** for collision resolution.

Files Provided:

- `hashing.h`
- `hashing.c` – You need to implement the following functions:
 - `int hash_function(char BITS_ID[])`: Computes the hash value as the sum of ASCII values of the 13 characters in the `BITS_ID`, modulo `HASH_TABLE_SIZE`.
 - `void insert_into_hash_table(Student student)`: Inserts the student into the hash table using linear probing.

What to Submit:

Save all the **seven source files** in a folder. Rename the folder using the format:

`<YourFirstName>_<YourID>`

Upload the zipped folder as your final submission.